

1. 前言

我是一名一两年之前就报名了预学习的学员，此前一生一芯的学习是零零散散，挫败不断，始乱终弃。25年近年底的时候再次开启了这个学习计划，这次顺利地完成了全部预学习内容，作为一个第一次被折磨的放弃者，以及第二次完成入门的初学者，结合自己两次截然不同的学习经历，本文主要谈以下几点：

1. 如何高效入门学习一生一芯；
2. 课程讲义的反馈建议；
3. 对主题“提问的智慧”的个人看法。

2 如何高效入坑一生一芯

结论先行：

1. 不浮躁的心态，大把连续不被打断的专注时间。
2. 预课程知识与工具的掌握熟练度(尤其是Linux环境编程及GNU工具链，极其重要没有之一)
3. 学科视野与减法思维

为了让各位饱受折磨的小白更有代入感，回放一遍我当初入坑的心路历程：

总感觉课程主页的预学习讲义和PA讲义不说人话，资料东一榔头西一锤子，照着做遇到了各种迷之问题，各种冒红报错，正式任务还没开始就被各种环境依赖，命令配置折磨得痛苦不堪，各种linux命令和开源工具五花八门让人抓不住重点困惑万分，遇到新的问题就不知道从哪下手如何定位分析，一动手写和设计更是一团浆糊。这就是我第一次入坑ysyx真实体验，相信只要不是本身有基础的大佬，小白在入坑时都会有上述类似的情景。

由于我的知识储备和认知并未升级，所以第二次我仍然面临上述困难和痛苦，但稍微改观的是上面提到的三点，让我有了对这些困难的应对手段，因为自己淋过雨，所以想结合个人经历给遭遇同样痛苦状态的初学者撑把伞。

当你觉得预学习阶段都痛苦不堪时，大概是以下几方面出现了问题：

1. 你的前置基础知识体系和工具链没有补齐
2. 你没有认真看讲义/教材/手册/源码/报错提示的每一个字
3. 你大概率处于浮躁状态，学习任务经常被一些琐事打断。
4. 断断续续中途放弃了很久没有坚持。

2.2 预备知识与工具

虽说ysyx是面向全体人群的开源芯片项目，项目里也列举了许多预学习阶段的必备资源，但我仍认为有一些知识和工具的重要程度在预学习阶段完全被低估而有些被夸大，过多的学习资料只会让初学者抓不住重点陷入茫然困惑，而且这些知识和工具的熟练度将极大程度地影响初学者的入门体验，我将以重要程度依次列出最为重要的一些知识体系和工具。

2.2.1 Linux环境编程

如果预学习阶段只能开设一门课程，对无基础的初学者而言，Linux环境编程及其GNU工具链的使用，这个重要程度远在所有预学习知识之上。

编程启蒙本该从命令行交互开始，然而许多高校编程语言的培养计划是教学生在IDE上点击run/debug，把背后本该展示的细节给隐藏了，导致我哪怕到了工作只会点击IDE的那个run/debug键，至于背后发生了什么我尝试过了解但总是会遭到引擎底层关于编译、链接、运行时库层面的阻隔，虽然读过CSAPP总体上知道它们是怎么个关系，但是缺少了来自Linux环境编程这一工具的视角，导致对这些涉及到运行时底层调用时总是盲人摸象不知所然进而望而生畏，不知道如何进行更深入的分析排查。直到后来系统自学了《Linux命令行与shell脚本编程》，我才算是真正获得了一种全新的系统视角。

Linux环境编程之所以这么重要主要有两点原因：

首先是熟悉命令行交互后，初学者就会树立正确高效的编程观念，通过命令交互亲手把源码用工具编译成汇编码，又是被哪些工具链接为可执行文件，全程操作系统内核提供了怎样的运行时环境支持，最终把文本变成目标ISA上的01比特串整个一过程是全程操作交互的，得来的理解自然比书本上那简短几句概念板书来得直观百倍。每当环境依赖出问题，或者程序出现了一堆冒红的trace后不再手足无措网上搜答案问AI，而是会尝试自己定位分析错误是在哪一环节。

其次，无论是GNU内置工具还是第三方开源工具以及tcl脚本本身就有共性，这套共性就是早期源于Unix系统的基本命令，管道，文件，信号，正则、脚本的通用符号规则，掌握它们后对一些新的工具就容易更快上手。

2.2.3 数字电路基础/verilog语言

数字电路基础的扎实程度直接影响后续对E5讲义中关于综合，仿真，布线，以及各种开源EDA工具的理解和使用，不仅仅组合逻辑和时序逻辑的设计，从顶层芯片到晶体管，晶体管工艺参数，动态特性，功耗，时序，面积，verilog的掌握程度直接影响你后续电路设计的正确性以及效率，反过来数字电路基础的扎实程度直接影响verilog仿真综合语义的理解。

2.2.2 GNU工具链

make,gcc,gdb,git这些内置工具也是极其重要的，但直接一上来就去看手册大概率会不知所然，别沮丧这是很正常的。因为工具是生来就是用来解决问题和提升效率的，如果你觉得工具晦涩难用，那么一定是你还没遇到这个工具要解决的问题场景，或者是对问题的理解还不到位。随着学习的深入，会越发发觉这些工具的强大以及熟练掌握它们的重要性，去自发地接受并学习它们。

2.2.4 C语言

个人认为初学阶段C语言的掌握反而不是那么的重要，初学阶段只需要树立一点编程基本概念即可（我的C语言基础就是大一期末考试水平没写过什么东西），完成概念的快速扫盲并在一些容易踩坑的地方留个心眼，即便我后面因为C语言不熟练，做nemu的PA时付出了痛苦代价，但我仍然坚持C语言对初学者的重要程度远不如上面提到的知识和工具重要，C语言和GNU工具一样，它们的重要程度和精进要求会随着遇到的问题被逐渐巩固加深。

2.2.5 计算机体系结构

这个是ysyx的灵魂核心课程，然而我认为这也不属于初学者领域，预学习阶段树立基本概念了解即可，在第一颗单周期CPU没有跑通之前，它都属于可有可无的课程知识，只有进阶到更高的分支预测准确率，更高的Cache命中，更快的访存和IO时，它的重要性才会逐渐凸显。

2.1 切勿浮躁，专注坚持

最小完成原则，事情未开始前，切勿完美主义，这种完美主义是拖延和挫败的绝佳温床，事情都还没开始就指望完美是不切实的期待，不切实的期待除了打击初学者信心，当天学习计划被一些不可抗拒的因素打断时，脑子就不要想着全部完成了，但更不要想完全不完成，。一个功能总觉得无从下手时，就大刀阔斧地砍，砍到不能再特殊不能再简单为止，，回想第一次开启那会儿还在工作

这个时代专注力和创造力正在变得越来越稀缺，各个互联网app为每个用户创建了数字标签和画像，给定体量身地投放精准的内容，流量时代夸大的封面配图和大字号标题从来不是信息的本源，无时无刻在侵蚀人的专注力并散播焦虑，物理隔绝低质信息源，为了学习而学习，本就不多的下班时间，刷个b站知乎，回个微信消息，开几把游戏，本来规定的三小时学习计划被各种中断，大脑这个CPU切换上下文的代价很大的(至少对我这笨比来说是的)，很多时候专注力和晶体管的阈值电压一样，某些任务不达到一定专注度和时长相当于白干。相信坚持的力量，不积跬步无以至千里。

2.3 认真阅读关键信息

很多时问题答案并不存在于互联网，而是赤裸裸地躺在报错的trace信息，讲义的某行字，教材的某张插图，文档手册的注脚，如果心态浮躁很容易忽略这些信息，转而选择最低效的互联网引擎搜索和问AI(很多时候问AI未必低效，但这会减少你独立缜密思考问题的机会)。

预学习阶段我踩过最大的坑就是F6阶段运行一生一芯logo时，所有指令都测过了办法用尽都没找出问题，最后发现电路没问题，是我没认真按照讲义要求把镜像文件加载到RAM导致浪费4天时间在调试电路的正确性。

看到密密麻麻的trace和log输出时不要慌，它们反而是指向问题根源的最佳依据。

自己埋头搭Logisim电路或者写模拟器太久没头绪时不要硬磕，翻开教材把模块框架梳理进自己脑海去实现，遇到问题再回头校对

此外按照ysyx始终倡导的原则RTFW,RTFM,RTFSC，网上的技术博客只能借鉴参考，最权威的一手资料永远是官方文档、教材和源码，我就在F6实现RV32I指令时，就因为参考了本中文译本浓缩版小册子，搞错了一条指令的功能码导致调试了许久，这么痛的领悟会让你感受到F字母的分量。

2.4 减法思维

思维阻塞想不明白时不要死磕，而是学会做减法非必要模块统统砍掉，从最简单最最特殊最笨比的实现入手，跑通最小子功能后，再把此前砍掉的内容逐渐加回来，这个过程更容易抓住本质

3 课程讲义反馈

结论先行：

1. E2的verilog学习内容移至F6，可以考虑为F7，但新增与F6内容的联动。
2. E3学习资料引入书籍《Linux命令行与shell脚本大全》
3. F3阶段增加CMOS工艺局限性和先进工艺半导体介绍的思考部分
4. F4可增加与自动控制学科的思考联动
5. F5/E5学习资料引入branch education有关芯片架构和芯片制造动画
6. E5阶段引入插件TerosHDL介绍
7. E5讲义内容太长，可以考拆分为C语言硬件仿真模型、verilator使用以及verilog仿真事件队列模型单独作为一节。后半的verilog综合语义,yosys综合工具和后端工具iEDA全流程作为一节

真心感谢对讲义进行升级的各位老师同学们，你们给我这种笨比降低了学习门槛，领略到了课程本该有的乐趣。作为一个刚结束E阶段预学习的菜鸟回看23.06讲义，如果让我以纯新手视角再出发，旧版讲义的体验着实不太好，它缺少了两块最关键的认知拼图——C代码到二进制的编译过程，以及RTL到电路实现的流程。不得不说即便我现在完成了预学习，如果纯按照旧版讲义内容自学的话很容易让人困惑，我会对旧版讲义从nvboard突然就引入verilator和yosys之类的工具感到万分不解，因为中间省略了太多动机和细节。

趁着对最新版讲义的记忆还在，提几点课程的学习体验和课程反馈建议如下：

1. 新版讲义增加了F阶段，其中每个小节必做题做完后都是充满惊喜，本以为自己对数字电路还算有基础，认真完成F3阶段的必做题后才发觉以前的理解非常浅薄，必做题过程中引发的思考探索，让我对锁存器、触发器，时序，晶体管特性，补码，电路行为与仿真差异的根源有了新的理解。**建议F3加入如今COMS工艺半导体的瓶颈局限，以及先进工艺相关的思考题和拓展部分，还有工业上对多位加法器的实现。**
2. F4部分对计算机状态模型的讲解让人印象非常深刻，一个从未注意到的视角，由于我专业是自动化，计算机被抽象为状态模型后发现最后都是在设计控制系统。**建议引入一些与其他学科的思考联动**
3. F5完美诠释了最小指令集，最简计算机系统，真正意义上的hello world。**这部分可以引入Youtube博主branch education中关于架构相关的精美动画视频**
4. F6的miniRV,让我对内存对齐，读写接口访问，指令电路设计有了基本理解,**这部分建议加入一些关于logisim高效调试方法。另外本节后半段其实可以鼓励学生从繁琐的logism手工连线转向HDL语言描述了，完全可以将E2对verilog的学习引入到本节或本节后**
5. E3阶段**作为一个鸟哥私房菜,missing semester教程都看过的初学者，强烈建议在E3阶段将《Linux命令行与shell脚本编程大全》作为学习资料甚至必学内容，这本书是做到了真正的入门易懂**
6. E4阶段精华，从C到二进制，再到自己手写miniRV软件模拟器，期间不断重构调试自己写的模拟器收获巨大。
7. E5更是精华中的精华，从一个最简单的C语言程序引入硬件仿真模拟，再无缝引入verilator仿真工具，以及后面的yosys综合和iEDA开源工具，其中子濠哥对verilog综合仿真语义，仿真器事件队列模型，芯片从设计到后端制造的全流程讲解更是鞭辟入里，让人印象十分深刻。最后的数电实验必做题收获也很大，它对数字电路设计和的verilog更系统的检验。**强烈建议本章引入TerosHDL插**

件，这个工具生成的RTL图辨识度更好且对仿真综合以及HDL的工程开发支持更完善，另外可以在本节穿插一些商业EDA巨头及其前沿仿真综合技术的资料，以及引入branch education有关现代芯片制造业全流程的动画视频

4. 提问的智慧

在接触一生一芯前就接触过“黄鸭调试法”，传达的理念基本一致，当你遇到任何难以调试的Bug和困难准备向他人寻求帮助之前，用**准确扼要**的语言先向桌面上的小黄鸭公仔(小黄鸭只是个指代，它可以是任何实体)问这几个问题：

1. 你想做什么；
2. 过程中你遇到了怎样的问题和困难；
3. 对此你已经进行了怎样的思考和尝试；
4. 尝试的结果如何，你接下来的推论猜测有哪些？

许多人在严格按照这四部曲操作后答案已经不言而喻自行解决，即便照做后问题仍然没有思路悬而未决，在贴上以上背景信息后再求助，资深专业的回答者会根据详尽的信息给出高效的回答。自己不动脑直接伸手问他人并不是勤学好问，反而是懒惰愚蠢(在极客社区看来甚至是缺乏教养)的表现。无论是小黄鸭调试法还是提问的智慧推荐的两篇文章，始终在传达相似的理念，一个合格的工程师应勤于思考，耻于不加思考的提问。专业与业余的鸿沟就是思维认知习惯，这个过程中你有了思考和尝试，直接问AI确实很高效，但交给它们的应该是一些重复琐碎和查字典的活，但决不能让它来代替人的独立思考、分析和创造那部分工作。

道理其实一直都懂得，但刻意练习并运用这个方法还是在ysyx的训练中给予了我认知和习惯上都有了很大的改观，遇到问题第一时间不是像以前一样复制粘贴丢搜索引擎和AI，而是先尝试分析报错提供的信息线索，多用工具去调试验证，陌生函数命令就man | grep，陌生开源工具就认真看工具的README、example里的makefile,以及官网的issue，RTFW,RTFSC,RTFM始终告诫我们应尽量接近信息的本源而不是他人的二次加工解读。除了底层学科视野的补全之外，这种思维给了人一种面临未知困难时的信心底气。

毫不夸张地讲，ysyx这一个半月的学习经历带给我的认知改观比前面二十年接收的教育还要多，从被动接受化为主动获取，从认知恐惧转换为求知满足，它倡导的提问方式和独立解决问题的思维是比任何课程知识都要宝贵的认知。